Ders 10 Çalışma Özeti

Ders 10 Çalışma Özeti

Genel Konular

- Divergent Change ve Shotgun Surgery (ekrandan, 00:10:00)
  - İkisi de "değişikliklerin farklı noktalarda yapılmasının gerekmesi" kusurudur. Gereksinim değişince ya da bir hata düzeltilince kodda değişiklik gerekir; iyi bir tasarımda bu değişiklikler mümkün olduğunca az yerde yapılır.
  - Divergent Change: bir gereksinim değişikliği için aynı sınıfın birçok farklı metodunda değişiklik yapılmasıdır. Örnek: yeni bir veritabanı ile çalışmak gerekince bir sınıfın 3 metodu değişir. Strategy kalıbı kullanılmış olsaydı yeni veritabanı için yeni bir sınıf eklemek yeterdi.
  - Shotgun Surgery: değişikliğin birden fazla sınıfın birden fazla noktasına yayılmasıdır; Divergent Change'den daha kötüdür. Örnek: kredi veren bir finansal programda yeni bir kredi türü gelince 2 sınıftaki 5 metodun değişmesi.
  - Hocanın benzetmesi: sınıf şemasında değişen yerleri işaretlediğinizde noktalar bir çifte (av tüfeği) atışı gibi geniş alana dağılıyorsa Shotgun Surgery vardır; ameliyat bıçakla yapılır. Divergent Change'de bıçak tektir ama aynı organın birçok yerine dokunulur.
  - Çözüm: Extract Class (değişen metotlar ve bu metotların ağırlıklı kullandığı üye alanlardan yeni bir sınıf çıkarılır) ya da Move Method / Move Field (bu üyeler daha uygun, mevcut bir sınıfa taşınır).
- Primitive Obsession (ilkel tiplerin aşırı kullanımı) (ekrandan, 00:16:30)
  - Yapısal programlamadan gelenler ilkel tipleri aşırı yoğun kullanır. Hoca "obsession" (takıntı) kelimesini "aşırı kullanım" diye çevirdi.
  - Farklı biçimleri tek tek tarif etmek yerine her birini düzelten refactoring eylemleri incelenir: Replace Data Value with Object, Replace Type Code with Class, Replace Type Code with Subclasses, Replace Type Code with State/Strategy, Replace Array with Object. Extract Class ve Introduce Parameter Object da daha nesneye yönelik bir program için kullanılabilir.
- Replace Data Value with Object (ekrandan, 00:20:00)
  - Bir üye alan, hele ki bu alanla ilgili çeşitli davranışlar varsa, bir sınıfla simgelenmelidir.
  - Örnek: `Order` sınıfında `customer: String` → `Order` ile `Customer` (`name: String`) arasında çokluğu 1 olan bir ilişki.
  - Bu durum veritabanı odaklı düşünmekten (string'in bir müşteri tablosuna yabancı anahtar olması) kaynaklanabilir. Müşteri başlı başına bir varlıktır. Ayrılmazsa belli bir müşterinin tüm siparişlerini bulmak gibi işlemler sipariş listesinde sıralı arama gerektirir ve `Order` sınıfı aşırı büyür.
  - Hoca slayttaki ilişki yönünü eleştirdi: bir müşteri birden fazla sipariş verdiği için parça-bütün ilişkisinde elmasın `Customer` tarafında olması gerektiğini düşündü ve kendine not aldı.
- Replace Type Code with Class (ekrandan, 00:24:40)
  - Yapısal programlamadaki sabit değişken kullanma alışkanlığıyla ilgilidir.
  - Örnek: `Person` içinde `0`, `A`, `B`, `AB` adlı dört `public static final int` sabit ve `bloodGroup: int`. Sonra: `Person` → (1) `BloodGroup`, sabitler `BloodGroup` türünden ve `BloodGroup` içinde.
  - Sakınca: `bloodGroup` üye alanına 5 gibi geçersiz bir değer atanabilir.
  - En iyisi doğrudan `enum` kullanmaktır (kan grubu, small/medium/large beden gibi sabit

ve değişmez seçenekler).
- Replace Type Code with Subclasses (ekrandan, 00:30:00)
  - Kalıtım kullanılması gereken yerde tip kodu kullanılmışsa, her metotta `switch(_type)` ya da `if ... else if` zincirleri görülür.
  - Önce: `Employee` içinde `_type` ve `ENGINEER = 0`, `SALESMAN = 1`, `MANAGER = 2`; `work()` metodu `switch` içerir. Sonra: soyut `Employee` ve ondan türeyen `Engineer`, `Salesman` sınıfları, her biri kendi `work()` gerçeklemesiyle.
  - Önceki halde `case default` varsa `Employee` soyut yapılmaz; genel çalışan türü `Employee` sınıfında kalır.
  - Bu düzenleme Replace Conditional with Polymorphism'e de benzer.
- Replace Type Code with State/Strategy (ekrandan, 00:32:20)
  - Kalıtım bir nedenle doğrudan kullanılamıyorsa: `Employee` → (1) `Employee Type`, ondan türeyen `Engineer` ve `Salesman`.
  - Avantajı: bir çalışanın türü çalışma anında değiştirilebilir (mühendis satış departmanına geçebilir). Alt sınıf çözümünde ise yeni bir `Salesman` nesnesi oluşturup bilgileri kopyalamak ve eskisini silmek gerekir; nesne kimliği (object identity) kaybolur ve iş daha masraflıdır. Bu esneklik gerekmiyorsa düz çok biçimlilik yeter.
- Replace Array with Object (00:34:29)
  - Farklı türden verileri bir iki boyutlu dizide ya da aynı indisle eşleşen paralel dizilerde tutmak yerine bir sınıf oluşturulur ve o sınıf türünden nesne dizisi kullanılır.
  - Hocanın örneği: bir ligde takım adı, puanı ve averajı. İki boyutlu dizide hepsi `String` olarak tutulursa her işlemde sayıya dönüştürmek gerekir; üç ayrı dizide ise aynı indisin aynı takıma ait olduğunu bilmek gerekir. Çözüm: `Team` (ya da `Score`) gibi bir sınıf.
  - Yapısal dillerde de bu iş için `record` (Pascal) ya da `struct` (C) vardır.
- Feature Envy (özellik kıskançlığı) (ekrandan, 00:40:00)
  - Bir metot ait olduğu sınıfın üyelerinden çok başka sınıfların üyelerine erişiyorsa bu kusur vardır. "Çok", o koddaki ortalama (ya da %20 gibi bir eşik) farklı sınıf üyesi erişim sayısının üstünde bir sayıdır.
  - Sorumlulukların doğru atanmadığının göstergesi olabilir. Bağlaşım ilkesi gereği başka sınıflardaki nesnelere erişmek gerekir; hiçbir sınıf yalnız kendi metotlarını kullanmaz, öyle olsaydı uyum ilkesi olumsuz etkilenirdi.
  - Strategy ve Visitor kalıpları özellik kıskançlığı olarak yorumlanmamalıdır.
  - Çözüm: Move Method; kıskançlık metodun yalnız bir kısmındaysa Extract Method ile o kısım çıkarılıp kıskanılan üyelerin bulunduğu sınıfa taşınır.
- Video kiralama örneğinde ardışıl refactoring (Fowler'ın Refactoring kitabının 1. bölümü) (01:01:42 - 02:00:54)
  - Sınıflar: `Customer` (müşteri, `statement()` ile makbuz üretir), `Rental` (kira kaydı, kaç gün kiralandığı), `Movie` (film, fiyat kodu: `REGULAR`, `NEW_RELEASE`, `CHILDRENS`). Bir müşterinin n kira kaydı vardır, her kira kaydı bir filmle ilişkilidir.
  - Başlangıçta `Customer` 57 satır, diğerleri 16 ve 22 satır; `statement()` 17. satırdan 56. satıra kadar yaklaşık 40 satırlık bir metottur ve içinde `switch` vardır. Bu yüzden işe `Customer` ile başlanır.
  - Her adımdan sonra JUnit test kümesi (`TestAll`) koşturulur; testler hiç değişmeden geçer.
  - Adımlar: Extract Method (`amountFor`), değişken adlarını değiştirme (`thisAmount` → `result`, parametre `each` → kira kaydını anlatan bir ad), Move Method (`amountFor` → `Rental.getCharge()`), Replace Temp with Query (`thisAmount` yerine `each.getCharge()`), sık kiralama puanı için `Rental.getBonus()`, `Customer.getTotalCharge()` ve `getTotalBonus()`, `getCharge`/`getBonus` hesabının `Movie`'ye taşınması, `Rental.getMovieTitle()`, `addFooter` ile altbilginin ayrı metoda çıkarılması.
  - Son adım (switch yerine çok biçimlilik) yapılmadı, yalnız tartışıldı.
- Anti-pattern'lere yönlendirme (01:56:11)
  - Kod kusurları küçük ölçeklidir. Birden fazla sınıfı kapsayan, tasarım kalıbı ölçeğinde kusurlar "anti-pattern" (karşıt kalıp) konusudur. Üç gruba ayrılır: yazılım geliştirme, yazılım mimarisi ve yazılım proje yönetimi karşıt kalıpları.
- Birinci ara sınavın akademik soruları tartışıldı (02:01:16 - 02:08:49)
  - Komut satırı boru hattı (Decorator), Java arayüzü Template Method'daki üst sınıf yerine kullanılabilir mi (hayır), Composite ve Iterator kalıplarını tanıma. Ayrıntılar genişletilmiş notta.

Hocanın Özellikle Vurguladığı Kısımlar

- Değişiklik mümkün olan en az yerde yapılmalıdır
  - Divergent Change ve Shotgun Surgery bunun ihlalidir; böyle bir noktaya gelindiyse tasarımı toparlamak gerekir.
- `switch`/`if-else if` zinciri kalıtım eksikliğinin işaretidir
  - Hoca dersin başında da söylediğini hatırlattı: kendinizi `switch case` ya da `if else if` kullanırken görüyorsanız büyük olasılıkla kalıtım ve çok biçimlilik kullanmanız gerekiyordur.
- Refactoring dış arayüzü değiştirmez, testleri değiştirmez
  - Video kiralama örneğinde her adımda test kümesi değişmeden koşturuldu. Son adımın (yeni `Movie` alt sınıfları) yapılmamasının nedeni birim testlerinin değişecek olmasıdır.
- İş, verinin olduğu yerde yapılmalı
  - Her sınıf mümkün olduğunca kendi verisiyle çalışmalı, ilişkide olduğu sınıfların verisiyle ilgili işleri o sınıflara bırakmalıdır.
- Yorum yerine anlaşılır kod
  - Yorumla ayrılmış kod parçaları ayrı metotlara çıkarılır, iyi ad verilmiş metot yorumun yerini alır.
- Sequence diagram, sınıf şemasında görünmeyen bağlaşımları gösterir
  - `Customer`'dan `Movie`'ye doğrudan çağrının kaldırıldığı sınıf şemasında değil, sequence diagram karşılaştırmasında görülür.

Kısa Tekrar Notları

- Divergent Change: bir değişiklik → bir sınıfta birçok metot. Shotgun Surgery: bir değişiklik → birçok sınıfta birçok metot.
- Çözüm: Extract Class, Move Method, Move Field.
- Primitive Obsession → Replace Data Value with Object, Replace Type Code with Class/Subclasses/State-Strategy, Replace Array with Object.
- `Order.customer: String` → `Customer` sınıfı.
- Sabit `int` kan grubu kodları → `BloodGroup` sınıfı ya da `enum`.
- `Employee._type` + `switch` → `Engineer`, `Salesman` alt sınıfları; tür çalışma anında değişecekse State/Strategy.
- İki boyutlu `String` dizisi → `Team` nesnelerinin dizisi.
- Feature Envy: başka sınıfın üyelerine kendi üyelerinden çok erişen metot → Move Method ya da Extract Method.
- Strategy ve Visitor Feature Envy sayılmaz.
- Video kiralama: `statement()` içindeki `switch` → `amountFor` → `Rental.getCharge()` → `Movie.getCharge(days)`.
- Sık kiralama puanı: her kira 1 puan; yeni çıkan film 1 günden fazla kiralandıysa 1 puan daha.
- `getTotalCharge()` ve `getTotalBonus()` listeyi ayrı ayrı dolaşır; liste bir yerine üç kez dolaşılır.
- Son adım: soyut `Movie` ve `MovieNewRelease`, `MovieRegular`, `MovieChildrens` alt sınıfları.

Detaylı Açıklamalar

- Replace Type Code with Subclasses kodu (ekrandan, 00:30:00):

```
/*** ÖNCE: ***/
class Employee {
  private int _type;
  static final int ENGINEER = 0;
  static final int SALESMAN = 1;
  static final int MANAGER = 2;
  Employee (int type) {
    _type = type; }
  public void work( ) {
    switch(_type) {
```

```
      case ENGINEER:
        //do something
        break;
      case SALESMAN:
        //do something
        break;
      case MANAGER:
        //do something
        break;
    }
  }
}

/*** SONRA: ***/
abstract class Employee {
  public abstract void work(){};
}
public class Engineer extends
    Employe {
  public void work( ) {
    //do something as an engineer
  }
}
public class Salesman...
```

- Ders sonrası not: Slayttaki `public abstract void work(){};` satırı Java'da derlenmez, çünkü soyut bir metodun gövdesi olamaz. Doğrusu `public abstract void work();` olmalıdır. Ayrıca `extends Employe` satırında `Employee` adı eksik yazılmıştır.
- Video kiralama örneğinin başlangıç hali, `Customer.statement()` (ekrandan, 01:06:30, 01:24:15):

```
public String statement( ) {
    double totalAmount = 0;
    int frequentRenterPoints = 0;
    Enumeration<Rental> rentals = _rentals.elements();
    String result = "Rental Record for " + getName() + "\n";
    while (rentals.hasMoreElements()) {
        double thisAmount = 0;
        Rental each = rentals.nextElement();
        //determine amounts for each line
        switch (each.getMovie().getPriceCode()) {
        case Movie.REGULAR:
            thisAmount += 2.0;
            if (each.getDaysRented() > 2)
                thisAmount += (each.getDaysRented() - 2) * 1.5;
            break;
        ...
        }
        // add frequent renter points
        frequentRenterPoints ++;
        // add bonus for a two day new release rental
        if ((each.getMovie().getPriceCode() == Movie.NEW_RELEASE)
                && each.getDaysRented() > 1) frequentRenterPoints ++;
        //show figures for this rental
        result += "\t" + each.getMovie().getTitle()+ "\t" +
        String.valueOf(thisAmount) + "\n";
        totalAmount += thisAmount;
    }
    //add footer lines
    result += "Amount owed is " + totalAmount+"\n";
    result += "You earned " + frequentRenterPoints
```

```
            + " frequent renter points";
    return result;
}
```

- Ücret kuralları (`Movie.getCharge(int days)` içindeki `switch`, ekrandan, 01:20:00, 01:46:30):
  - `REGULAR`: 2.0; 2 günden fazlaysa her ek gün için 1.5 eklenir.
  - `NEW_RELEASE`: gün sayısı × 3.0.
  - `CHILDRENS`: 1.5; 3 günden fazlaysa her ek gün için 1.5 eklenir.
- Toplamlar için sorgu metotları (ekrandan, 01:40:00):

```
public double getTotalCharge() {
    double result = 0;
    for( Rental aRental : _rentals )
        result += aRental.getCharge();
    return result;
}
public int getTotalBonus() {
    int result = 0;
    for( Rental aRental : _rentals )
        result += aRental.getBonus();
    return result;
}
```

- Birikimci `result` değişkeni kümülatif toplam için kalmak zorundadır; geçici değişken `statement()`'tan kalktı ama bu metotlara geçti.
- `Rental` köprü metotları (ekrandan, 01:50:00):

```
public double getCharge( ) {
    return _movie.getCharge(_daysRented);
}
public int getBonus( ) {
    return _movie.getBonus(_daysRented);
}
```

- Önerilen son tasarım (ekrandan, 01:53:30; slayt "Refactoring Örneği – 10. Müdahale"):

```
Customer                        Rental                          <<abstract>> Movie
+ statement()                   - _daysRented : int             + getCharge(daysRented : in
+ getTotalCharge() : double     + getCharge() : double          + getBonus(daysRented : int
+ getTotalBonus() : int         + getBonus() : int
                                + getMovieTitle() : String

Customer 1 ---- * Rental   (Customer'dan Rental'a yönlü)
Rental   1 ---> 1 Movie
Movie <|-- MovieNewRelease, MovieRegular, MovieChildrens
```

- `switch/case` ve `if` deyimleri yerine kalıtım ve soyut metotlarla çok biçimlilik kullanılır. Hoca bunun en baştan yapılması gerektiğini, ama yeni sınıflar ortaya çıkacağı ve birim testleri değişeceği için geriye uyumluluk isteniyorsa yapılmayabileceğini söyledi. Replace Type Code with Strategy de bir seçenektir; testler daha az ölçekte değişir.
- **Not:** İsterseniz bu dersin altyazı (.srt) dosyasını Gemini Notebook gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.